

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH.V.RAMAYOGI	Reg. No.:	192424319
Section / Batch:	SLOT-B/AIDS-24-28	Date:	23-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I CH.V.RAMAYOGI/192424319(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH.V.RAMAYOGI

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____

Date: _____

Date: _____

Date: _____

Q7. Minimum Spanning Tree – Network Design

1. Introduction

An Internet Service Provider (ISP) needs to connect multiple cities using communication cables. Each possible connection between two cities has a different cable installation cost. The main objective is to connect **all cities with the minimum possible total cable cost**, while ensuring that every city can communicate with every other city.

This problem can be solved efficiently using a **Minimum Spanning Tree (MST)**. An MST is a spanning tree of a connected, weighted, undirected graph with the minimum possible total edge weight.

Two popular greedy algorithms for finding an MST are:

1. **Prim's Algorithm**
2. **Kruskal's Algorithm**

Both algorithms use a greedy strategy, but they construct the MST in different ways.

2. Industry Problem Understanding

Consider an ISP planning a network between six cities:

- Chennai
- Bangalore
- Hyderabad
- Mumbai
- Pune
- Delhi

The possible cable connections and their installation costs are:

Connection	Cost
Chennai – Bangalore	10
Chennai – Hyderabad	15
Bangalore – Hyderabad	12
Bangalore – Mumbai	20
Hyderabad – Mumbai	18

Hyderabad – Pune	25
Mumbai – Pune	8
Mumbai – Delhi	30
Pune – Delhi	16
Hyderabad – Delhi	28

The objective is to select connections such that:

- Every city is connected.
- There are no unnecessary connections.
- No cycle is formed.
- The total cable cost is minimum.

For V cities, an MST always contains exactly:

[
 $V-1$
]
 edges.

For six cities:

[
 $6-1=5$
]

Therefore, the final network should contain **5 cable connections**.

3. Graph Representation

The network can be represented as a weighted graph:

[
 $G=(V,E)$
]

where:

- V = set of cities
- E = possible cable connections
- Weight of an edge = cable installation cost

For the given problem:

[
V=6
]

and the edges represent possible communication links between cities.

A possible minimum-cost network is:

Selected Edge	Cost
Mumbai – Pune	8
Chennai – Bangalore	10
Bangalore – Hyderabad	12
Chennai – Hyderabad	15
Pune – Delhi	16

However, selecting both Chennai–Hyderabad and the Chennai–Bangalore–Hyderabad path would create a cycle. Therefore, the MST must be constructed carefully.

The actual MST obtained by applying the algorithms is:

Selected Connection	Cost
Mumbai – Pune	8
Chennai – Bangalore	10
Bangalore – Hyderabad	12
Chennai – Hyderabad	15
Pune – Delhi	16

The edge Chennai–Hyderabad creates a cycle with Chennai–Bangalore and Bangalore–Hyderabad, so it must **not** be selected. Therefore, the correct MST is:

MST Connection	Cost
Mumbai – Pune	8
Chennai – Bangalore	10
Bangalore – Hyderabad	12
Pune – Delhi	16
Hyderabad – Mumbai	18

Minimum Total Cost

$$[$$
$$8+10+12+16+18=64$$
$$]$$

Therefore, the minimum cable installation cost is:

$$[$$
$$\boxed{64}$$
$$]$$

4. Kruskal's Algorithm

Kruskal's algorithm is an edge-based greedy algorithm.

It works by:

1. Sorting all edges according to increasing cost.
2. Selecting the cheapest edge.
3. Checking whether adding the edge creates a cycle.
4. If no cycle is created, adding the edge to the MST.
5. Continuing until $V-1$ edges are selected.

Sorted Edges

Edge	Cost
Mumbai – Pune	8
Chennai – Bangalore	10
Bangalore – Hyderabad	12
Chennai – Hyderabad	15
Pune – Delhi	16
Hyderabad – Mumbai	18
Bangalore – Mumbai	20
Hyderabad – Pune	25
Hyderabad – Delhi	28
Mumbai – Delhi	30

Selection Process

Step 1: Select Mumbai–Pune = 8

Step 2: Select Chennai–Bangalore = 10

Step 3: Select Bangalore–Hyderabad = 12

Step 4: Chennai–Hyderabad = 15 is rejected because it creates a cycle:

Chennai → Bangalore → Hyderabad → Chennai

Step 5: Select Pune–Delhi = 16

Step 6: Select Hyderabad–Mumbai = 18

Now five edges have been selected for six cities, so the algorithm stops.

Kruskal MST Cost

[

$8+10+12+16+18=64$

]

Therefore:

[

$\boxed{\text{MST Cost}=64}$

]

5. Prim's Algorithm

Prim's algorithm is a vertex-based greedy algorithm.

It starts from any vertex and repeatedly selects the minimum-cost edge that connects a vertex already inside the MST to a vertex outside the MST.

Starting from Chennai

Initially:

[

$\text{MST}=\{\text{Chennai}\}$

]

The cheapest connection from Chennai is:

Chennai – Bangalore = 10

Add Bangalore.

Next, the cheapest edge connecting the current MST to an unvisited city is:

Bangalore – Hyderabad = 12

Add Hyderabad.

Next:

Hyderabad – Mumbai = 18

Add Mumbai.

Next:

Mumbai – Pune = 8

Add Pune.

Finally:

Pune – Delhi = 16

Add Delhi.

The five selected edges are:

Edge	Cost
Chennai – Bangalore	10
Bangalore – Hyderabad	12
Hyderabad – Mumbai	18
Mumbai – Pune	8
Pune – Delhi	16

Total:

[
10+12+18+8+16=64
]

Therefore:

[
 $\boxed{\text{MST} \setminus \text{Cost}=64}$
]

Both algorithms produce the same minimum total cost.

6. Comparison of Prim's and Kruskal's Algorithms

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Vertex-based	Edge-based

Strategy	Grows one tree	Builds forest and combines components
Starting point	Requires a starting vertex	No specific starting vertex
Main data structure	Priority Queue / Min Heap	Disjoint Set / Union-Find
Edge sorting	Not necessarily required	Required
Cycle detection	Naturally avoided by visited set	Explicitly required
Suitable graph	Dense graphs	Sparse graphs
Time complexity with heap	$O(E \log V)$	$O(E \log E)$
Implementation	Relatively simple	Requires Union-Find
Practical use	Network expansion	Large sparse networks

7. Time Complexity Analysis

Prim's Algorithm

Using an adjacency list and binary min-heap:

[
 $O(E \log V)$
]

where:

- V = number of vertices
- E = number of edges

Using an adjacency matrix, the complexity is:

[
 $O(V^2)$
]

Therefore, the choice of implementation depends on the graph representation and network density.

Kruskal's Algorithm

Kruskal's algorithm first sorts all edges.

Sorting takes:

[
 $O(E \log E)$
]

Union-Find operations take approximately:

[
 $O(E \alpha(V))$
]

where $\alpha(V)$ is the inverse Ackermann function and grows extremely slowly.

Therefore, the overall complexity is:

[
 $O(E \log E)$
]

Since:

[
 $E \leq V^2$
]

Kruskal's algorithm is generally very effective for sparse graphs.

8. C++ Implementation of Kruskal's Algorithm

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
struct Edge {  
    int u, v, weight;  
};
```

```
class DSU {  
    vector<int> parent, rankValue;
```

public:

```
DSU(int n) {
    parent.resize(n);
    rankValue.resize(n, 0);

    for (int i = 0; i < n; i++)
        parent[i] = i;
}

int find(int x) {
    if (parent[x] == x)
        return x;

    return parent[x] = find(parent[x]);
}

bool unite(int a, int b) {
    a = find(a);
    b = find(b);

    if (a == b)
        return false;

    if (rankValue[a] < rankValue[b])
        swap(a, b);

    parent[b] = a;

    if (rankValue[a] == rankValue[b])
        rankValue[a]++;
}
```

```
        return true;
    }
};
```

```
int main() {
```

```
    int V = 6;
```

```
    vector<Edge> edges = {
        {0, 1, 10},
        {0, 2, 15},
        {1, 2, 12},
        {1, 3, 20},
        {2, 3, 18},
        {2, 4, 25},
        {3, 4, 8},
        {3, 5, 30},
        {4, 5, 16},
        {2, 5, 28}
    };
```

```
    sort(edges.begin(), edges.end(),
        [](Edge a, Edge b) {
            return a.weight < b.weight;
        });
```

```
    DSU dsu(V);
```

```
    int totalCost = 0;
```

```
    int edgesSelected = 0;
```

```

cout << "Minimum Spanning Tree:\n";

for (auto edge : edges) {

    if (dsu.unite(edge.u, edge.v)) {

        cout << edge.u << " - "
            << edge.v << " : "
            << edge.weight << endl;

        totalCost += edge.weight;
        edgesSelected++;

        if (edgesSelected == V - 1)
            break;
    }
}

cout << "\nMinimum Cable Cost = "
    << totalCost << endl;

return 0;
}

```

Expected Output

Minimum Spanning Tree:

3 - 4 : 8

0 - 1 : 10

1 - 2 : 12

4 - 5 : 16

2 - 3 : 18

Minimum Cable Cost = 64

Here the city numbers represent:

0 = Chennai

1 = Bangalore

2 = Hyderabad

3 = Mumbai

4 = Pune

5 = Delhi

9. Suitable Data Structures

For Prim's Algorithm

The recommended data structures are:

- Adjacency list
- Min-priority queue
- Boolean visited array
- Distance/key array

A priority queue allows the algorithm to efficiently identify the lowest-cost connection.

For Kruskal's Algorithm

The recommended data structures are:

- Edge list
- Sorting algorithm
- Disjoint Set Union (DSU)
- Parent array
- Rank/size array

DSU is particularly important because it efficiently determines whether adding an edge would create a cycle.

10. Recommended Tools

The following tools can be used to develop and evaluate the network optimization solution:

Tool	Purpose
------	---------

C++	Algorithm implementation
Visual Studio Code	Code development
GCC/G++	Compilation
Git/GitHub	Version control
Python	Performance analysis and visualization
NetworkX	Graph and MST experimentation
Excel	Cost and result analysis
Matplotlib	Graph visualization

For an academic implementation, **C++ with VS Code and GitHub** is sufficient. For larger industry datasets, Python with NetworkX can be used for experimentation and visualization.

11. Practical Network Design Interpretation

The MST provides a cost-efficient network backbone.

In the example, the ISP has six cities and ten possible cable connections. Installing all ten connections would result in unnecessary infrastructure expenditure.

The MST uses only five connections while still ensuring that all six cities are connected.

The minimum cost is:

$$\boxed{64 \text{ units}}$$

If the cost unit represents ₹1 lakh, then:

$$64 \times 1,00,000 = ₹64,00,000$$

Thus, the estimated cable installation cost would be **₹64 lakh**.

The MST also avoids unnecessary cycles. This makes it suitable for designing a basic low-cost network infrastructure.

However, a real ISP may not always use only an MST because redundancy is important. If one cable fails in an MST, the network can become disconnected. Therefore, after constructing the minimum-cost backbone, an ISP may add selected redundant links to improve fault tolerance and reliability.

12. Prim vs Kruskal – Industry Recommendation

For a network containing many cities but relatively fewer possible connections, **Kruskal's algorithm** is a strong choice because it works efficiently with an edge list and is particularly suitable for sparse graphs.

For a dense network where many connections are possible between cities, **Prim's algorithm** with a priority queue or suitable graph representation can be more appropriate.

Recommended Approach

For the given ISP problem:

Kruskal's Algorithm + Disjoint Set Union

is recommended because:

1. The network can be represented naturally as a list of possible cable connections.
2. Cable costs can easily be sorted.
3. DSU efficiently prevents cycles.
4. The algorithm is suitable for sparse networks.
5. It provides a clear cost-based selection process.

13. Performance Evaluation

For the given network:

- Number of cities = 6
- Number of possible connections = 10
- MST edges = 5
- Minimum cable cost = 64

Both Prim's and Kruskal's algorithms produce:

[
\boxed{64}
]

This validates that the network can be connected using five carefully selected links without unnecessary cycles.

For larger networks, the performance can be evaluated using:

- Number of vertices

- Number of edges
- Execution time
- Memory consumption
- Total network cost
- Number of selected connections

A useful experimental approach is to generate networks with increasing numbers of cities and compare the execution time of Prim's and Kruskal's algorithms.

14. Key Issues Identified

The important issues in the ISP network design problem are:

1. **High infrastructure cost** – unnecessary cables increase installation expenses.
2. **Network connectivity** – every required city must remain connected.
3. **Cycle prevention** – redundant connections can increase cost.
4. **Scalability** – the algorithm must work efficiently for large networks.
5. **Fault tolerance** – a pure MST has no redundancy.
6. **Dynamic network requirements** – new cities and connections may be added later.
7. **Real-world constraints** – geographical distance, terrain, maintenance cost, and installation restrictions may need to be considered.

15. Advantages of MST-Based Network Design

- Minimizes total connection cost.
- Connects all required locations.
- Avoids unnecessary cycles.
- Provides an efficient network backbone.
- Easy to implement using greedy algorithms.
- Scales well for large graph-based problems.
- Can be extended with additional constraints for real-world network planning.

16. Limitations

Although MST minimizes total cable cost, it does not automatically optimize every real-world network requirement.

For example:

- It does not provide redundant paths.
- It does not consider traffic capacity by default.
- It does not consider latency.
- It does not automatically account for geographical obstacles.
- It assumes that the edge costs correctly represent the objective.

Therefore, an actual ISP may combine MST with additional network optimization techniques.

17. Conclusion

The ISP city-connection problem can be effectively modeled as a **Minimum Spanning Tree problem**. Prim's and Kruskal's algorithms are greedy optimization techniques that construct an MST while minimizing the total connection cost.

For the given six-city network, both algorithms produce an MST with:

[
 $\boxed{5\text{edges}}$
]

and a minimum total cable cost of:

[
 $\boxed{64\text{units}}$
]

Kruskal's algorithm uses sorting and Disjoint Set Union, making it particularly suitable for sparse networks. Prim's algorithm grows the network from a selected starting city and is effective when using adjacency structures and priority queues.

For practical ISP network planning, MST provides a strong cost-optimization foundation. However, additional links may be required after MST construction to provide redundancy, reliability, capacity, and fault tolerance.

Thus, MST algorithms provide an efficient and practical solution for **cost-optimized network design**, while real-world ISP deployment should combine the MST result with reliability and operational constraints.